the computer and the programming language itself. One of the reasons Lisp does that is because it is designed to be extensible. Read on for more details.

There is no substitute for a good book while learning Lisp. Start with a free resource. I recommend the following resources that are available online:

1. *On Lisp*—by Paul Graham, available on his website: *http://www.paulgraham.com/onlisp.html*
2. *Practical Common Lisp*—by Peter Seibel, available at *http://www.gigamonkeys.com/books/*
3. *Successful Lisp*—by David B Lamkins, available at *http://psg.com/~dlamkins/sl/contents.html*

The following text snippets are from Paul Graham's website, which carry extracts of his books, 'On Lisp' and 'ANSI Common Lisp'.

## Bottom-up programming

Lisp is designed to be extensible; it lets you define new operators yourself. This is possible because the Lisp language is made out of the same functions and macros as your own programs. So it's no more difficult to extend Lisp than to write a program in it. In fact, it's so easy (and so useful) that extending the language is standard practice. As you're writing your program down toward the language, you build the language up toward your program. You work bottom-up, as well as top-down.

Almost any program can benefit from having the language tailored to suit its needs, but the more complex the program, the more valuable bottom-up programming becomes. A bottom-up program can be written as a series of layers, each one acting as a sort of programming language for the one above. TEX was one of the earliest programs to be written this way. You can write programs from the bottom-up in any language, but Lisp is by far the most natural vehicle for this style.

Bottom-up programming leads naturally to extensible software. If you take the principle of bottom-up programming all the way to the topmost layer of your program, then that layer becomes a programming language for the user. Because the idea of extensibility is so deeply rooted in Lisp, it makes for the ideal language to write extensible software.

Working bottom-up is also the best way to get reusable software. The essence of writing reusable software is to separate the general from the specific; bottom-up programming inherently creates such a separation. Instead of devoting all your efforts towards writing a single, monolithic application, you devote part of your effort to building a language, and part to writing a (proportionately smaller) application on top of it. What's specific to this application will be concentrated in the topmost layer. The layers beneath will form a language for writing applications like this one—and what could be more reusable than a programming language?

## Rapid prototyping

Lisp allows you to not just write more sophisticated programs, but to write them faster. Lisp programs tend to be short—the language gives you bigger concepts, so you don't have to use as many. As Frederick Brooks (best known for his book 'The Mythical Man-Month') has pointed out, the time it takes to write a program depends mostly on its length. So this fact alone means that Lisp programs take less time to write. The effect is amplified by Lisp's dynamic character: in Lisp, the edit-compile-test cycle is so short that programming happens in real-time.

Bigger abstractions, and an iterative environment, can change the way organisations develop software. The phrase 'rapid prototyping' describes a kind of programming that began with Lisp -- in Lisp, you can often write a prototype in less time than it would take to write the spec for one. What's more, such a prototype can be so abstract that it makes a better spec than one written in English. And Lisp allows you to make a smooth transition from prototype to production software. When Common Lisp programs are written with an eye for speed and compiled by modern compilers, they run as fast as programs written in any other high-level language.

All of this obviously means that you can now spend less time working, and finally take your family out for that dinner you've been promising them, for the last three years—happy boss, happy family. It also means that you have a very valid excuse to stick your head in your copy of *LFY* each month—you're reading that article that not only teaches you how to become a better programmer, but also how to make the world a happier place!

## Questions answered, and a response to cynical feedback

I received many questions about the Lisp programming language after the first article was published in the June 2011 edition of *Linux For You*. Fortunately, David B Lamkins, author of the book 'Successful Lisp', has provided comprehensive answers to most of these questions on the following page: *http://www.psg.com/~dlamkins/sl/chapter01.html*. The questions he answers are:

1. I looked at Lisp earlier, and didn't understand it.
2. I can't see the program for the parentheses.
3. Lisp is very slow compared to my favourite language.
4. No one else writes programs in Lisp.
5. Lisp doesn't let me use graphical interfaces.
6. I can't call other people's code from Lisp.
7. Lisp's garbage collector causes unpredictable pauses when my program runs.
8. Lisp is a huge language.
9. Lisp is only for artificial intelligence research.
10. Lisp doesn't have any good programming tools.
11. Lisp uses too much memory.
12. Lisp uses too much disk space.
13. I can't find a good Lisp compiler.